Final Report for Syslab

Towards Generalized Ground-based Multi-Agent Robotic Search from Graph Neural Networks

Rocco Zhang, Rishabh Kumaran

May 27, 2025

Yilmaz - Period 3

Table of Contents

Abstract

Multi-agent robotic systems trained on reinforcement learning have the potential to improve efficiency in deterministic or single-agent situations. In the context of search and rescue (SAR), coordinating multiple autonomous agents to locate a target in a complex, partially observable environment presents both operational and communicational challenges. Our research aims to create a multi-agent reinforcement learning (MARL) framework in which a team of robots collaboratively searches for a stationary target in a continuous environment containing obstacles. Additionally, to encourage cooperation and greater spatial awareness, a graph neural network (GNN) is employed to represent inter-agent communication. Training is conducted using a centralized learning, decentralized execution paradigm, and our preliminary results suggest that our algorithm has significantly better results than a deterministic approach.

1. Introduction

In the aftermath of a natural disaster, the rapid location of survivors is one of the most essential challenges faced by emergency response teams. Research shows that the probability of survival for individuals trapped or lost in disaster zones declines quickly after 72 hours, with the survival rate dropping to 20% after three days [1]. This study suggests that there is an urgent need for automated search and rescue systems capable of conducting wide-area searches quickly and efficiently, especially in environments where human access is not feasible.

Motivated by this real-world issue, we aim to develop a multi-agent system trained through reinforcement learning (RL) to collaboratively search through for a target in a simulated environment. The initial idea for this problem originates from studying limitations in single-agent and deterministic search strategies, which often struggle in uncertain environments or conditions. Multiple autonomous agents, in comparison, operating concurrently can reduce search time, but also introduce many new challenges of coordination, communication, and decision-making, combined with the fact that the environment is partially observable.

What really interests us in this project is the intersection of both real-world constraints with complex algorithmic challenges. MARL offers a new promising framework for distributed decision-making, but it also requires more sophisticated ways in which agents can share relevant information and make a decision using each other's positions and observations. In typical MARL algorithms, communication may either be absent, predefined, or limited to explicit messages, which may not scale well to different environments.

The novelty of our algorithm lies in integrating a GNN into the MARL framework to model inter-agent communications based on spatial relationships and sensor data. At each timestep, a dynamic graph is constructed, where nodes represent agents, and edges represent communication links between agents. The GNN then processes this graph to produce a global embedding that captures the structure of the environment, enabling agents to make more aware decisions even when the environment is only partially observable. This approach allows agents to effectively simulate cooperative behavior, even under decentralized execution.

There are two main benefits for our approach. First, it improves the efficiency and reliability of multi-agent search operations by enabling better communication. Second, it provides a scalable framework adaptable to unseen spatial constraints commonly found in SAR scenarios, such as a collapsed building. By addressing both the theoretical and practical challenges of decentralized communication, our work contributes to the development of intelligent systems capable of supporting critical humanitarian missions where time is an invaluable resource.

2. Related Works & Background

2.1 Early Applications of MARL for Target Search

One of the first papers I looked at was the research presented by Qin et al., investigating the problem of multi-agent cooperative target search using reinforcement learning [2]. Their framework focuses on agents with limited sensing and communication capabilities, searching for stationary targets in a discrete environment. Each agent maintains a local probability map representing belief over target locations, which is updated through Bayesian fusion with nearby agents. This probability map is used to inform an RL policy trained via a Deep Q-Network. While their approach does not use GNNs and uses a Deep Q-Network, which only operates on discrete action spaces, it contributed several essential ideas, including the use of inter-agent communication and the design of reward functions that incentivize exploration. These principles are highly relevant to our project, where agents must balance exploration, observations, and cooperation.

2.2 Comparison of Multi-agent Reinforcement Learning Training Frameworks

The research done by Abdulghani et al. provides a comparative evaluation of several prominent MARL algorithms, such as QMIX, Value Decomposition Networks (VDN), Multi-Agent Actor-Attention-Critic (MAA2C), and MAPPO within the StarCraft Multi-Agent Challenge environment [3]. Their results demonstrate that while QMIX and VDN achieve higher success rates in certain cooperative scenarios, MAPPO remains competitive, particularly in environments that benefit from its policy-gradient-based architecture and CTDE compatibility. Although this study does not incorporate graph-based modelling, it reveals MAPPO's potential for scalable and stable learning in multi-agent systems. The empirical findings support our choice of MAPPO for agent training, especially given its capacity to handle decentralized execution without sacrificing performance.

2.3 Integration of GNN with MARL architecture

Research conducted by Goeckner et al. proposed a novel GNN-based MARL architecture known as Multi-Agent Graph Embedding-based Coordination (MAGEC), which was explicitly designed to support coordination among agents in environments with partial observability and communication disturbances. Their work addresses key shortcomings of traditional multi-agent coordination methods, especially how fragile they can be during real-world deployment. In the MAgeC framework, agents and their communication links are modelled as nodes and edges in a dynamic graph, allowing the GNN to process and aggregate spatial information. The architecture integrates MAPPO within a centralized training and decentralized execution paradigm, with a centralized critic and a GNN-based actor capable of interpreting the graph-based inputs. The actor employs a "neighbor scoring" mechanism that ranks the utility of neighboring nodes to make discrete movement decisions. This design supports robust multi-agent patrolling across large-scale graphs and shows successful performance under noisy conditions. The modeling of agents as a communication graph, the use of a GNN for policy optimization, and the MAPPO training framework is very similar to our project, which similarly aims to achieve distributed coordination and exploration in a partially observable environment. The two key differences between our project and this paper is that we intend to operate on a continuous action space, and the goal of the agents is target discovery, rather than patrolling.

3. Applications

As stated earlier, the primary motivation for this project lies in its potential application to search and rescue operations. When conditions are dangerous, or simply inaccessible to human responders, deploying autonomous systems to locate survivors can dramatically improve response times and safety. The ability for multiple agents to cooperatively search large or obstructed areas, adapt to different conditions, and cooperate with other agents offers numerous advantages over traditional solutions. However, beyond SAR missions, this MARL framework has potential applications in a wide range of other fields where distributed agents must collaborate in an environment filled with obstacles.

One such situation is in a warehouse and inventory management. In large, complex warehouse settings, multiple agents working together are increasingly used for tasks such as inventory scanning or item retrieval. Applying a similar MARL and GNN-based coordination strategy could improve efficiency and optimize search paths, particularly in high-density or changing storage environments.

Additionally, this system could even be adapted for monitoring an environment or for exploration and mapping tasks. Applications might include tracking wildlife in expansive, obstacle-dense habitats, inspecting the structure in a hazardous industrial site, or coordinating agricultural monitoring over a large farmland where multiple ground-based and aerial robots could work together efficiently.

Another interesting application of our research lies in defense and security operations, where decentralized, cooperative multi-agent systems would be highly useful for tasks such as perimeter surveillance, intruder detection, or for reconnaissance in hazardous terrain. The ability for autonomous agents to reason using their surroundings and maintain awareness of the position of peers in real-time, without actually requiring centralized control, is particularly advantageous in a variety of settings.

Finally, this framework could contribute to the development of intelligent transportation systems, where vehicles, autonomous or otherwise, can share information about hazards and traffic conditions to make more coordinated decisions. Integrating GNN-based reasoning with local observations could enhance navigation safety and efficiency in congested urban environments or emergency evacuation scenarios. In summary, even though this work is primarily focused on SAR applications, its core ideas of decentralized multi-agent reinforcement learning, partial observability, and graph-based communication has broad potential in many different fields.

4. Methods

4.1 Multi-Agent Reinforcement Learning

In reinforcement learning, an agent interacts with an environment by observing its current state, taking an action, and receiving a reward, with the objective of learning a policy that will maximize the cumulative future reward over time. In multi-agent reinforcement learning, this framework is extended to include multiple agents interacting within a shared environment. Each agent independently observes a partial view of the environment, selects an action according to its own policy, and receives an individual or shared reward based on the resulting environmental transition.

In fully cooperative MARL tasks, such as multi-robot search and rescue, all agents have a common goal, meaning that the policy should be trained to encourage cooperation. However, coordination among decentralized agents with partial observability remains a fundamental challenge, as agents must learn policies that optimize their own performance while complementing the behavior of other agents, without directly having access to their observations, actions, or policies.

There are two primary learning paradigms commonly used in MARL. The first approach is independent learning, where each agent independently learns its own policy while treating other agents as part of the environment. The second approach is centralized training with decentralized execution (CTDE), where agents are trained using a centralized critic that has access to the joint observations and actions of all agents during training, but the actions are executed independently during testing using only their local observations. The CTDE paradigm is particularly well-suited for robotics where centralized computation and communication are

possible when training during the simulation phase, but are restricted or unreliable during training.

4.2 Multi-Agent Proximal Policy Optimization

Proximal Policy Optimization (PPO) is a widely-used actor-critic reinforcement learning algorithm known for its stability and sample efficiency in continuous action tasks. In PPO, an actor network develops the policy, while a critic network estimates the value function, which is used to compute the advantage estimate and guide the changes to the policy. The core idea of PPO is to constrain policy updates while clipping these changes, preventing large, destabilizing updates through a clipped objective function.

MAPPO extends PPO to the multi-agent domain while using the CTDE paradigm. In MAPPO, each agent maintains its own decentralized policy network that operates on local observations. Additionally, a centralized critic is used, which is shared across all agents during training, and operates on the joint observations and actions of all agents to determine a global value function. In MAPPO's implementation, the centralized critic estimates the expected return given the joint state and joint actions of all agents at each timestep. This critic is used only during training to compute advantage estimates, after which each agent's decentralized actor policy is updated separately. During execution, agents use only their local observation histories to select actions, without access to the centralized value function or other agent's policies.

4.3 Graph Neural Network & Integration

A key aspect of our research was the integration of a Graph Neural Network (GNN) into the multi-agent reinforcement learning framework to enable structured, indirect communication among agents. In multi-agent systems, especially under partial observability, coordination is challenging when agents rely solely on local observations. To address this, we model the multi-robot environment as a dynamic, relational graph at each timestep, capturing both the spatial relationships between agents. At each timestep, a dynamic graph is constructed, where nodes represent the agents, edges represent spatial relationships such as proximity and line-of-sight, and the node features include each agent's position, velocity, and sensor-derived information. Edge features include the distance between agents and line-of-sight indicators. Next, a graph neural network processes this graph using message-passing operations, where each node iteratively exchanges information with its neighbors along the graph edges. This enables agents to aggregate information about nearby teammates, obstacles, in a structured manner.

In this work, we employ a Graph Convolutional Network architecture, although other variants could be applied as well. The GNN updates node embeddings according to the following equation:

$$h_i^{(l+1)} = \sigma\left(\sum_{j \epsilon N(i)} W^{(l)} h_j^{(l)}\right)$$

Where $h_i^{(l)}$ is the feature vector node i at layer l, $N(i)$ is the set of neighbors of node i, $W^{(l)}$ is a learned weight matrix, and σ is the activation function.

After multiple rounds of message passing, the GNN outputs a final embedding for each node that encodes both local observations and contextual information from the neighboring nodes. A global graph-level embedding is then obtained by applying a readout function, such as a global mean or max pooling, over all of the node embeddings.

In the integrated framework, the GNN operates alongside the MARL policy learning process. Specifically, at each environment timestep, a graph representing the current state of the agents is constructed. The GNN processes this graph to a global embedding, which should capture relational information about agent positions, obstacles, and visibility. This global embedding is concatenated with each agent's local observation vector, to form an augmented observation. The augmented observation is then passed to the agent's policy network within the MAPPO framework.

During training, the centralized critic in MAPPO also conditions on the GNN-derived embeddings and joint actions to compute accurate value estimates. This improves the critic's ability to evaluate the cooperative value of actions in complex environments. During execution, each agent uses only its local observations and the graph embedding derived from information within its communication or sensing range. This design enables agents to learn decentralized policies while using structured communication through the GNN during training, preserving decentralized decision-making during testing.

4.4 Environment Simulation

The multi-agent search environment used in this project was implemented using the PettingZoo library, a Python package for creating standardized multi-agent reinforcement learning environments. The environment is designed as a continuous two-dimensional space populated with multiple autonomous agents, a stationary target, a set of stationary obstacles. The agents operate under partial observability constraints, relying on simulated sensors and limited-range communication to navigate the environment and cooperatively locate the target.

The simulated environment is defined by a bounded rectangular area with user-specified width and height parameters. Obstacles within the environment are modeled as rectangles with fixed dimensions, aligned with the axes of the environment. These obstacles are randomly placed at the start of each episode and remain stationary through the episode. Collision detection for both agents and obstacles are performed using geometric intersection operations between agent positions, sensor rays, and obstacle boundaries.

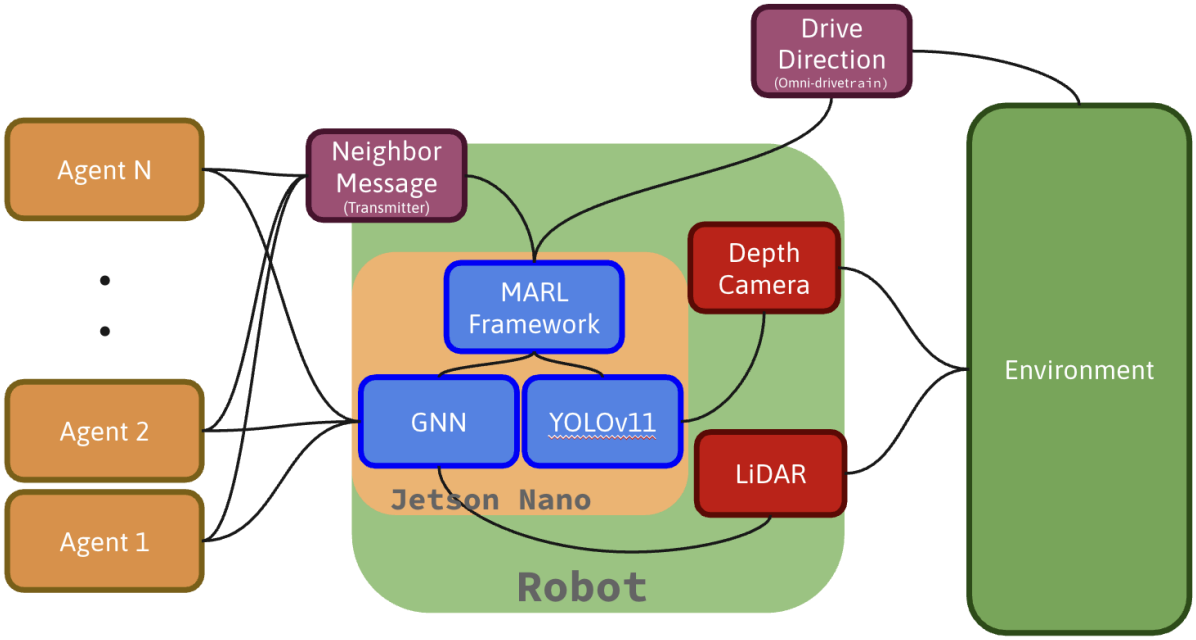
Each agent has four primary capabilities. The success range is a fixed radius around the agent within which if the target is within, the simulation ends in a successful discovery. The camera range is the range within which the agent can observe the target, when an unobstructed line of sight exists. The LiDAR range is the maximum distance the LiDAR rays are sent that allows for distance measurements to the nearest obstacles. The LiDAR system for each agent is

simulated by casting a series of equally spaced rays radiating outward from the agent's position at 4° increments, producing a total of 90 rays per full rotation. Finally, the communication range is the fixed radius defined as the maximum distance at which agents can share information or exchange messages, simulated via graph-based connectivity constraints.

To prevent redundant movements and encourage coverage of new areas, the environment maintains a record of points previously visited by each agent. This was implemented using a KDTree data structure, which enables efficient nearest-neighbor queries in a continuous space. At each timestep, an agent's current position is compared to the KDTree of visited points, and if it is farther than a specified minimum distance from all previously visited points, it is added to the tree. This data is used both for reward shaping, and for analysis of search coverage.

4.5 Systems Architecture

Figure 1



Looking at the system architecture, each of the agents draws observations from the environment using the depth camera and LiDAR scanner. The input of the depth camera is fed into the YOLOv11 model for target detection, and the output from the YOLO model is fed into the MARL training framework. Additionally, the observations from each agent created by the LiDAR scans, along with additional information, is fed into the GNN, which is then also fed into the training framework. The training framework is also directly informed by the messages from each independent agent. Finally, each agent interacts with the environment by their drive direction, which is computed through the policy created by the MARL training framework.

4.6 Packages Used

Throughout this project we used a number of packages and for a variety of purposes. The packages we primarily relied on are:

- PettingZoo: Used for the creation of a multi-agent environment.
- Pygame: Necessary for rendering the environment, so that visualization is much more straightforward.
- Stable Baselines: Package used for the training of our model.
- Supersuit: For the vectorization of the PettingZoo environment, so that it is compatible with Stable Baselines.
- Numpy: Necessary for computation and the handling of matrices.
- SciPy: Used to maintain the visited observation in the environment
- PyPlot: Used for the visualization of the results graphically.

5. Results

Figure 2

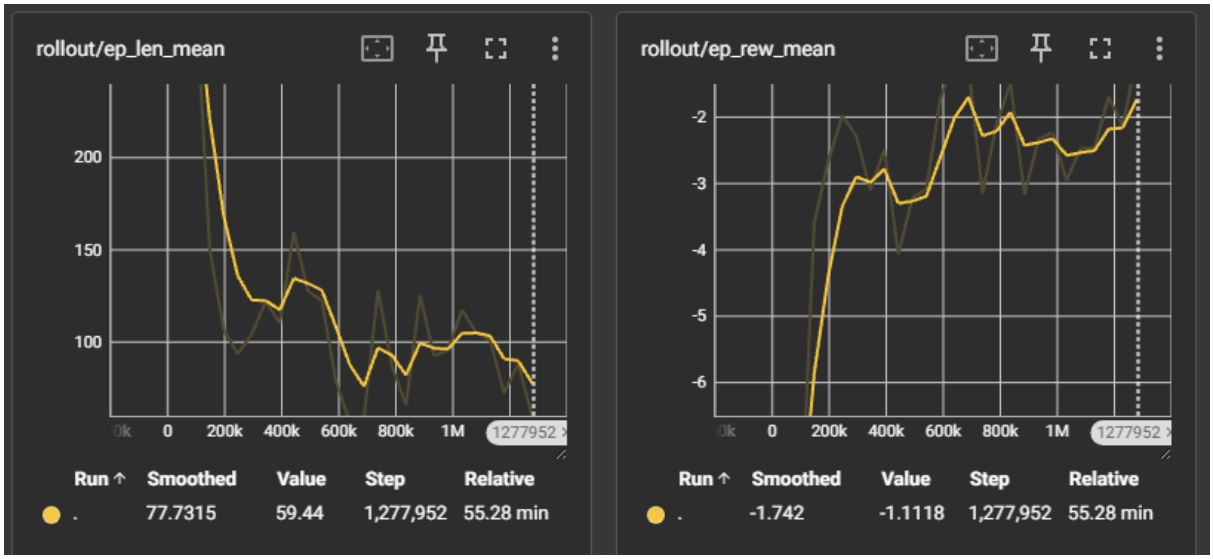


Figure 2 is a graph representing the metrics during training of the final and highest-performing GNN model. On the left is the episode length mean, or the average duration it takes the agents to find the target. On the right is the episode reward mean, or the average reward across agents for the whole episode. The graphs indicate relative convergence at the end of training and show significant improvements in our custom reward function. The final trained model is then the best model achieved.

Figure 3

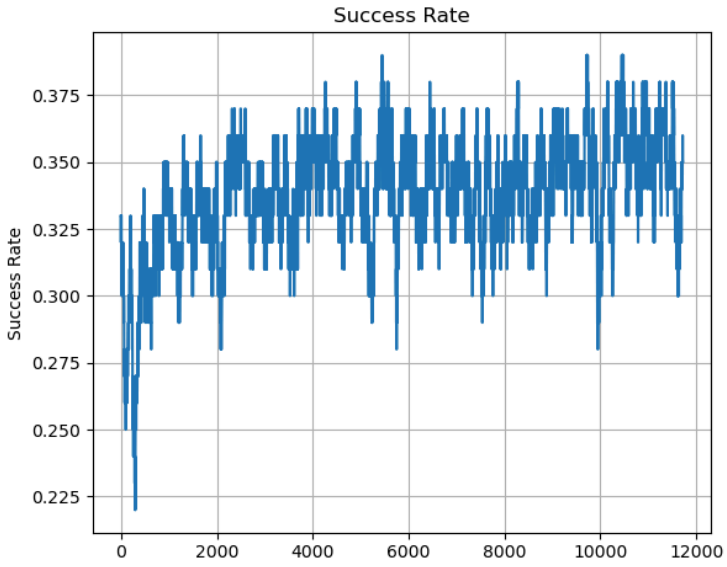


Figure 3 indicates the total rate of success versus the number of training episodes completed. We note that despite the apparent variance between episodes, the success rate has converged around 35%, far better than the initial performance nearer to 25%. When compared to a deterministic algorithm like floodfill, we observe the speed of success to be around 40% faster, despite better scalability and efficiency.

6. Limitations

Throughout our project we encountered a number of issues that limited the scope and the applicability of our results. The first issue that we faced was regarding the software present on the HiWonder JetAuto. The HiWonder JetAuto utilizes the Robot Operating System (ROS), a middleware between programs and hardware drivers. This has been implemented by the manufacturer. In order to begin interaction with the physical agent the roslaunch command must be called. However, we faced a unique issue with each agent. With the first agent, corruption of ROS led us to reinstall Python and ROS, yet afterwards, roslaunch runs into an error consistently when launching one of the nodes, making it impossible for us to operate ROS. On the other hand, running roslaunch on the second agent resulted in the command freezing after the step of "imu calibration". We believe that the robot was in this state prior to us receiving it, suggesting that resolution of this issue would require further understanding of the manufacturer's ROS system.

Beyond the physical application of our algorithm, there are a number of ways in which our theoretical algorithm was limited as well. First, the environment in which the agents operate is limited to a two-dimensional continuous space. While this simplifies the problem and reduces the computational complexity, it does not fully capture the spatial challenges present in real-world disasters, which often involve complex three-dimensional terrain or varied elevation. Extending the environment to three dimensions would introduce additional difficulties in sensing, communication, and navigation, and would likely require significant changes to both the

modelling of the environment, and the learning architecture, while also significantly increasing computational costs.

Another important limitation is the concern of computational resources. Although the centralized training process using MARL and GNNs is manageable in a controlled environment, scaling this approach to larger agent populations, more accurate or complex environments would require considerably more computational power and memory. Real-time execution on a policy stored on the physical agents presents even more challenges, as it would require a lightweight inference mechanism compatible with the limited resources on the agent.

Additionally, the current implementation assumes perfect communication within line-of-sight constraints and does not model realistic communication

Finally, another issue involving our algorithm is that although the simulated environment includes obstacles and constraints that create partial observability, it still remains as an abstraction of real-world conditions. Real situations are highly unpredictable, and could include irregularly shaped debris, unstable structures, and unpredictable external factors such as smoke, water, or simply poor visibility. The simulated sensors used by the agents, like the simulated LiDAR and camera detection, may not adequately reflect the random noise or failure of physical sensors in real-world environments.

7. Conclusion

In our work, we designed a novel multi-agent framework for search and rescue. Our specific situation involves ground-based, omnidirectional robots with a 2D LiDAR sensor for obstacle avoidance and a stereoscopic optical camera for target detection and localization. We leverage graph neural networks to create a decentralized and scalable policy to advance not just search and rescue, but also multi-agent coordination. By designing a reinforcement learning-based model, we hope to be able to integrate this manner of ground agent in coordination with other types, like drones, to assist in real-world search and rescue applications involving obstacles that are better suited for ground robots, such as rubble that may be best manipulated from the ground. Our work involved creating a 2D environment with unknown obstacles to train multiple agents to explore and coordinate in locating a target with an unknown location. By allowing agents to process input features with a convolution, we allow them to learn message representations to communicate with their neighbors most effectively, laying down the groundwork for a larger decentralized application where agents can move in and out of communication with each other, and allowing agents to act on their own. Due to the encountered obstacles, our work demonstrates promising results in simulation only, and we look forward to witnessing a physical application of the algorithm we designed.

8. Future Works

While this project establishes a foundation for multi-agent collaborative search using reinforcement learning and graph-based communication, several opportunities exist to extend upon our work and refine the framework in the future.

First, one of the most immediate and impactful extensions would be incorporating a third spatial dimension into the simulation environment. Real-world search and rescue missions frequently occur in multi-level structures or uneven terrain where elevation changes play an important role in navigation and visibility. Expanding the environment to three dimensions would introduce new challenges to the sensors, simulation of LiDAR, communication, and movement, and would more accurately reflect the real-world situations that are faced by disaster response teams.

A second direction our project could be expanded open would involve expanding the role of optical cameras beyond their current use for target detection. In realistic environments, cameras could provide valuable information about environmental obstacles and hazards that LiDAR alone cannot detect. Integrating image-based perception into the observation space, potentially through convolutional neural networks, could enable more advanced understanding of the environment for agents, and would improve both individual agent decision-making and team coordination.

Another idea that could be implemented in the future involves changing the policy training framework from MAPPO to an alternative multi-agent reinforcement learning algorithm. In particular, Value Decomposition Networks offer potential by decomposing joint value functions into individual agent components, which can improve the stability and learning efficiency while training. According to one study, when comparing a Value Decomposition Network, a Multi-Agent Advanced Actor-Critic algorithm, and a Multi-Agent Proximal Policy Optimization algorithm, in the SMAC environment, where agents both defend and attack against other agents, the Value Decomposition Network consistently had the greatest performance [3]. Comparing this algorithm against MAPPO within this environment would provide valuable insights into their relative strengths, particularly as the number of agents and the complexity of tasks increase.

Additional future work could focus on enhancing the realism of various aspects of the environment, like the communication constraints, for example. The current implementation assumes reliable, instantaneous communication within the radius, which oversimplifies real-world communication. Modelling limited bandwidth, communication delays, or message loss would introduce important real-world challenges and would allow for the testing of more realistic models.

In order to improve upon the results we obtained, one approach could be to use a curriculum learning technique to create a more flexible policy that would demonstrate greater success in more randomized environments. By exposing agents to a wider range of environment configurations or adding sensor noise conditions during training, the system becomes more resilient to unforeseen scenarios during deployment.

Finally, testing the trained policies on physical robots in a controlled experiment would serve as a critical validation step. While the algorithm may work theoretically, many issues can arise with the physical hardware. This transition from simulation to hardware would introduce practical challenges and inaccuracies, and would serve as an essential step in advancing this work towards readiness in real-world applications. While this project offers a promising foundation for the field of multi-agent cooperative search, there are still substantial opportunities for improvement, from improvements on the algorithm to physical deployment.

9. References

9.1 Citations

[1] Amini, M., Kachitvichyanukul, V., & Sirivipanich, V. (2022). Application of Soft Systems Methodology in solving disaster emergency logistics problems. International Journal of the Analytic Hierarchy Process, 14(1), 1–20. https://doi.org/10.13033/ijahp.v14i1.1050

[2] Qin, X., Li, X., Liu, Y., Zhou, R., & Xie, J. (2020). Multi-agent cooperative target search based on reinforcement learning. In *Journal of Physics: Conference Series*, *1549*(2), 022104. IOP Publishing. https://doi.org/10.1088/1742-6596/1549/2/022104

[3] Abdulghani, A. M., Abdulghani, M. M., Walters, W. L., & Abed, K. H. (2024). Performance evaluation of multi-agent reinforcement learning algorithms. Intelligent Automation & Soft Computing, 39(2), 338–352. https://doi.org/10.32604/iasc.2024.047017

[4] Goeckner, A., Sui, Y., Martinet, N., Li, X., & Zhu, Q. (2024). *Graph neural network-based multi-agent reinforcement learning for resilient distributed coordination of multi-robot systems*. arXiv preprint arXiv:2403.13093. https://doi.org/10.48550/arXiv.2403.13093

9.2 Github Repository

All of our work can be found at this Github repository. The repository contains our full codebase, our final presentation, along with this final paper.